



RAISE-Bot

— Hands-on ROS 2, AI Agents, and Greenhouse Robot Inspection



LLM Tool-Calling & MCP for Robotics

Student edition — tools, MCP, and safety, one infographic per idea

Professor Anis Koubaa • 14–16 July 2026



Husky A200
Base

Learning Objectives

By the end of this lecture, you will be able to:

- Distinguish **function-calling**, **ReAct**, and **MCP** — and pick the right one.
- Explain what **MCP** (Model Context Protocol) standardises and why it matters.
- Apply the **tool-design principles**: granularity, **idempotency**, recoverable errors.
- Build a **safety pipeline**: validation, dry-run, human-in-the-loop.
- Read the **live demo**: an LLM autonomously driving the sim robot.

Part 1

Three Ways to Connect Tools

Function Calling vs ReAct vs MCP

Three related concepts for tool-using AI systems



FUNCTION CALLING



- ✓ What it is: provider feature
- ✓ Model returns structured `tool_calls`
- ✓ Lives in: API request (`tools=[...]`)



REACT



- ✓ What it is: prompt pattern
- ✓ Interleaves reason + act
- ✓ Lives in: your prompt / loop logic



MCP



- ✓ What it is: protocol
- ✓ Standardizes how tools are exposed
- ✓ Lives in: a separate `server` the host connects to



HOW THEY FIT TOGETHER



They are not competitors.



ReAct = how the loop thinks



Function calling = how the model calls tools



MCP = how tools are packaged and shared



Day 1 lab: function-calling inside a ReAct-style loop.

MCP is the **next step** for reuse.

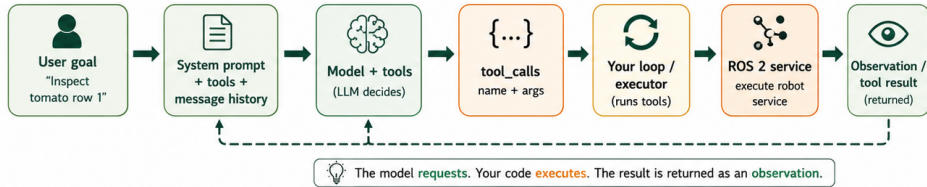


Function Calling in an Agent Loop

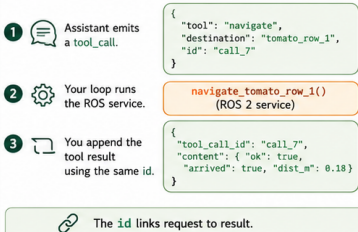
How the LLM, your loop, ROS services, memory, and observations work together



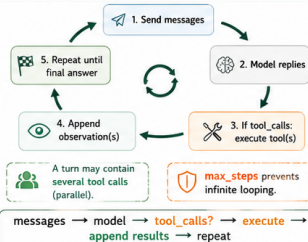
1 MAIN PIPELINE



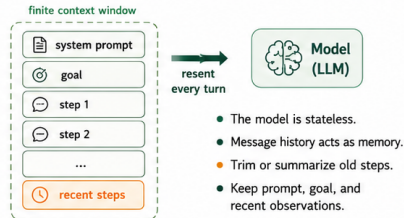
2 ANATOMY OF ONE TOOL CALL



3 WHOLE AGENT LOOP



4 MEMORY AND CONTEXT



Key idea: Agent = LLM + tools + loop + memory of recent observations



Function calling is the **mechanism**; observations **close the loop**.

Part 2

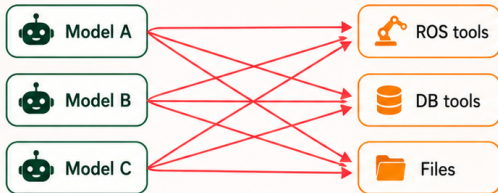
MCP — Write a Tool Once



Why MCP? The N×M Integration Problem

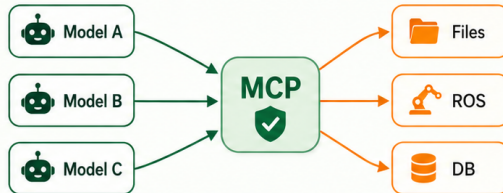
Without a standard, every model needs custom glue for every tool. With MCP, one protocol connects them all.

❌ Without a standard



every model × every tool = **custom glue**

✅ With MCP



write a tool **once**, any MCP client can use it



MCP = a **USB-C** for tools

one protocol so tools and models plug together without bespoke adapters.



Why it matters



less custom glue

eliminate N×M integrations



reuse across models

tools work with any MCP client

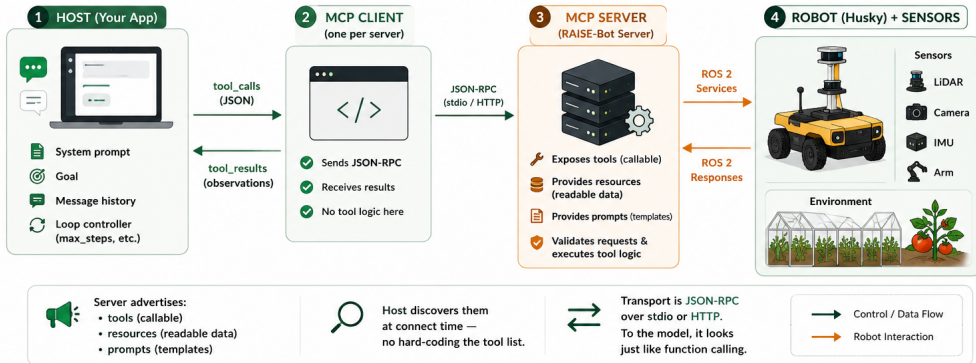


cleaner architecture

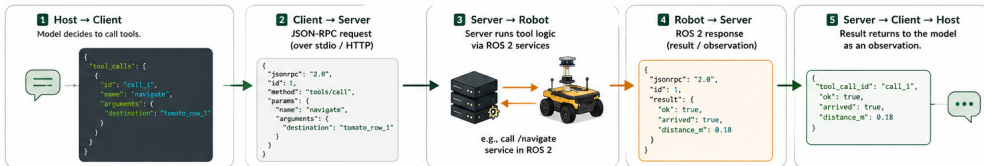
simpler, standard connections

MCP Architecture: Host, Client, Server

A clean, standard bridge between AI models and robotic capabilities.



WHAT HAPPENS INSIDE (AT A HIGH LEVEL)



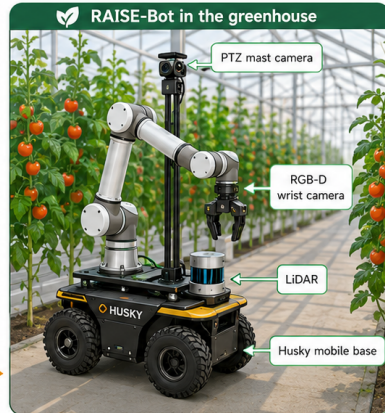
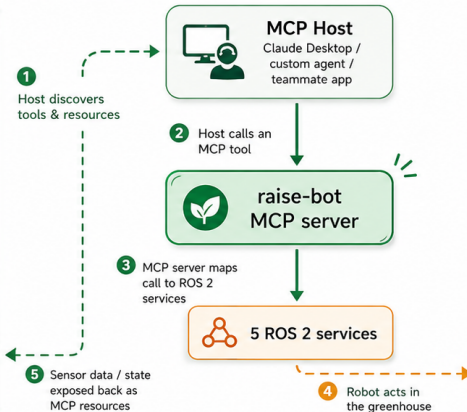
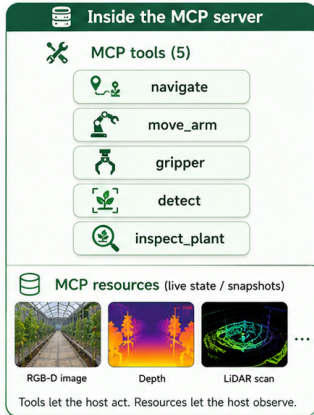
MCP = USB-C for tools: one standard protocol that lets any host connect to any server and immediately use its tools, resources, and prompts.



MCP for Robotics: The RAISE-Bot MCP Server

Wrap robot capabilities as MCP tools so any MCP host can control the robot

RAISE-Bot
Robotics for Agriculture



- ✓ **Reuse:** any MCP host can drive the robot
- Claude Desktop, custom agents, or teammate apps—plug in and go.



- ✓ **No need to import your robot code into every app**
- Keep robot code in one place. Expose capabilities via MCP.



- ✓ **Production packaging of robot capabilities**
- Tools, resources, and service mappings—versioned, tested, and ready for deployment.

Part 3

Designing Good Tools

1 Granularity

Pick the right size for a tool (Goldilocks rule)



TOO FINE

```
set_wheel_velocity(1, r)
```

⚠️ too low-level



JUST RIGHT

```
navigate(destination)
```

✅ one meaningful skill



TOO COARSE

```
inspect_whole_greenhouse()
```

⚠️ hides important decisions



Rule: one tool = one verb
the model would say out loud.

3 Recoverable Errors

Return structured results, not crashes.



Camera view
occluded

```
{  
  "ok": false,  
  "error": "occluded",  
  "hint": "re-aim PTZ  
           or move closer"  
}
```



Model reads the hint
→ tries another action



A tool should **never** crash the loop.

Tool Design for Agents

Four principles for designing good robot tools



RAISE-Bot

HUSKY IN THE GREENHOUSE



Good tools make better agents

clear verbs + safe repeats +
structured failures + sharp descriptions

2 Idempotency

Repeat should not change the outcome.



GOOD

```
navigate('row_1')  
x3 → still at row 1
```



RISKY

```
rotate_by(90deg)  
x3 → 270 deg
```



Prefer **absolute, goal-stated** tools
over **relative, incremental** ones.

4 Descriptions Steer the Model

The model picks tools from their descriptions.



BAD (vague)

```
navigate: moves the robot
```



GOOD (sharp)

```
navigate: Drive to a NAMED waypoint  
before inspecting a plant. Call BEFORE  
move_arm / inspect_plant.  
destination in {home, tomato_row_1,  
tomato_row_2, olive_grove}
```



Write descriptions **for the model**,
not for yourself.



Best practices:



meaningful skill



safe to retry



failure as data



clear tool descriptions

Part 4

Safety & the Live Agent Demo



RAISE-Bot Safety & Live Agent Demo



How safety gates protect the robot, and how the agent runs an autonomous inspection mission

1 Safety Layer 1 — Validation & Preconditions



Schema validation

Is the tool real? Are args in range?



Preconditions

Is the gripper empty? Is the arm stowed before driving?



Bounds

Is the target waypoint inside the greenhouse?



Check the action is well-formed and currently possible.



If a check fails, return a **structured error** + hint so the model **re-plans**.

3 Safety Layer 3 — Human-in-the-Loop



For **irreversible** or **high-stakes** actions, require confirmation.



approve



deny



Sensor reading = auto



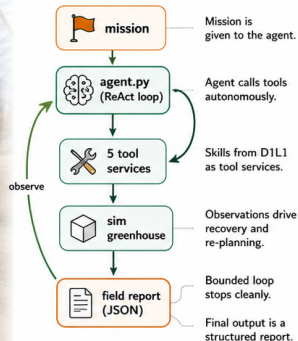
Harvest / cut = ask first

2 Safety Layer 2 — Dry-Run / Simulate First



- ✓ Test in simulation first
- ✓ Preview path / pose
- ✓ Catch collisions before hardware

4 Live Demo — An LLM Drives the Robot



Husky + UR5e in a tomato greenhouse



Autonomous inspection mission



Safety gates at every step



Traceable, reliable, repeatable



Key Takeaways



Validate before acting.



Simulate before hardware.



Ask humans for risky actions.



ReAct loop + tools enables autonomy.



Safe autonomy ends with a **structured report**.

Recap

Everything on One Poster

AGENTIC ROBOTICS WITH MCP

One Language Model. One Protocol. Real-World Robots.



LLM



TOOLS



LOOP



MCP
(USB-C for tools)



Why MCP? The N×M Integration Problem

Without a standard (N×M chaos)



⚠ Every model × every tool = custom glue.

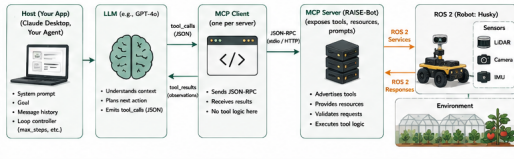
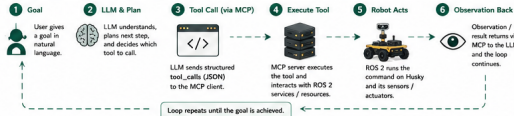
With MCP (N+M simplicity)



✓ Write a tool once, any MCP client can use it.

MCP = a USB-C for tools: one protocol so tools and models plug together without bespoke adapters.

MCP Pipeline End-to-End



Function Calling vs ReAct vs MCP

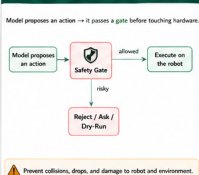
	What it is	Lives where
Function Calling	Model returns structured tool_calls.	In the API response (tools=[...])
ReAct	Prompt pattern of reasoning + acting in text.	In your prompt / loop logic.
MCP	Protocol that standardizes how tools are exposed.	A separate server the host connects to.

💡 They are not competitors:
ReAct = how the loop thinks + Function Calling = how it calls
MCP = how tools are packaged & shared

Tool Design (4 Principles)

- Granularity**
One tool = one verb the model would say out loud. ("Drive to row 1.")
- Idempotency**
Calling it twice has the same effect as once. (navigate("row_1") x3 = still row 1)
- Recoverable Errors**
Never crash the loop. Return structured result with "ok" or "error" + hint.
- Descriptions Steer the Model**
Clear tool descriptions + a strong system prompt guide correct choices and safe behavior.

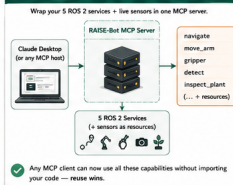
Safety Gate (Every Action First)



The Three Safety Layers

- Validation & Preconditions (Think)**
Check well-formedness and feasibility first.
• Schema valid? • Precondition ok? • Bounds ok?
- Dry-Run / Simulate First (Preview)**
Simulate in Gazebo (the lab) before real execution.
• Get planned path / pose
• Detect collisions
- Human-in-the-Loop (High Stakes)**
Ask for human confirmation for risky actions.
• Tier your tools by risk
• Confirmation is a tool too

MCP for Robotics: the RAISE-Bot Server



Legend

Green = Model/Logic
(Thinking, Planning)

Orange = Tools/Systems
(Acting, Executing)

Red = Safety/Protection
(Preventing Bad Things)

→ = Data / Control Flow ⇄ = Loop / Feedback



Bottom line: MCP lets many models work with many tools safely and consistently.
One protocol. Less glue code. More time building real robot capabilities.



Key Takeaways

- **Function-calling**, **ReAct**, and **MCP** are complementary: how it calls, how it thinks, how tools are packaged.
- **MCP** solves the $N \times M$ problem — write a tool once, any client uses it.
- Good tools are **verb-sized**, **idempotent**, and return **structured errors with hints**.
- Safety is a **pipeline**: **validate** → **dry-run** → **human-in-the-loop** for risky actions.

Questions?

Next: *Lab 1.2 — the agentic inspector* ([day1_02](#)).

Full slides + code walk-throughs are in the main lecture deck.